Laporan Monitoring and LB: K8S

Penyediaan dan Automasi Layanan

Nama Kelompok :

Muhammad Habibi (235150201111063)

Nadhif Rif'at Rasendriya (235150201111074)

UNIVERSITAS BRAWIJAYA

FAKULTAS ILMU KOMPUTER

2026

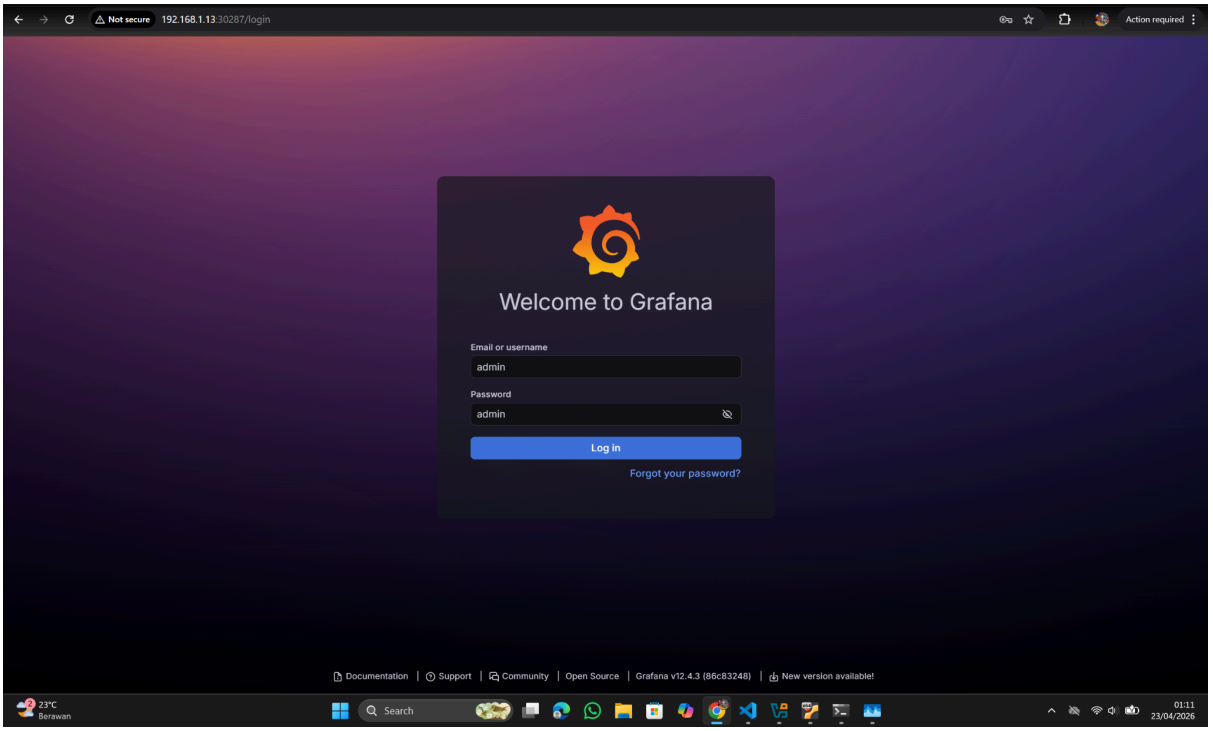
BAB 1 - Implementasi Sistem Monitoring

```
Every 2.0s: kubectl get pods -n monitoring -o wide                muhammad-habibi-VirtualBox: Thu Apr 23 02:17:03 2026

NAME                                                      READY   STATUS    RESTARTS      AGE   IP               NODE
  NOMINATED NODE   READINESS GATES
alertmanager-mon-stack-kube-prometheus-alertmanager-0     2/2     Running   0             95m   10.244.133.207   worker-2
  <none>           <none>
mon-stack-grafana-7b568555cb-sv8bn                        3/3     Running   2 (70m ago)   100m  10.244.97.199    worker-3
  <none>           <none>
mon-stack-kube-prometheus-operator-85c999c974-bdnmp       1/1     Running   1 (98m ago)   100m  10.244.97.198    worker-3
  <none>           <none>
mon-stack-kube-state-metrics-5d6cf447b8-f4xmj             1/1     Running   2 (19m ago)   100m  10.244.97.200    worker-3
  <none>           <none>
mon-stack-prometheus-node-exporter-9vhk8                  1/1     Running   5             100m  192.168.1.13     muhammad-habibi-virtualbox
  <none>           <none>
mon-stack-prometheus-node-exporter-df8gn                  1/1     Running   0             100m  192.168.1.12     worker-3
  <none>           <none>
mon-stack-prometheus-node-exporter-1x5hk                  1/1     Running   4             97m   192.168.1.11     worker-2
  <none>           <none>
mon-stack-prometheus-node-exporter-xt6kp                  1/1     Running   0             100m  192.168.1.10     worker-1
  <none>           <none>
prometheus-mon-stack-kube-prometheus-prometheus-0         2/2     Running   1             95m   10.244.133.208   worker-2
  <none>           <none>
```

Gambar 1. Status Pods pada Namespace Monitoring

Melalui eksekusi perintah ‘kubectl get pods -n monitoring -o wide’, dilakukan verifikasi terhadap kesiapan seluruh komponen monitoring stack. Hasil menunjukkan bahwa seluruh pod esensial meliputi Prometheus, Grafana, Alertmanager, dan Node Exporter, telah berada dalam status Running dan tersebar secara merata di seluruh node (Master dan ketiga Worker). Distribusi ini membuktikan keberhasilan konfigurasi high availability pada lapisan pemantauan klaster.



Gambar 2. Halaman Login Antarmuka Grafana

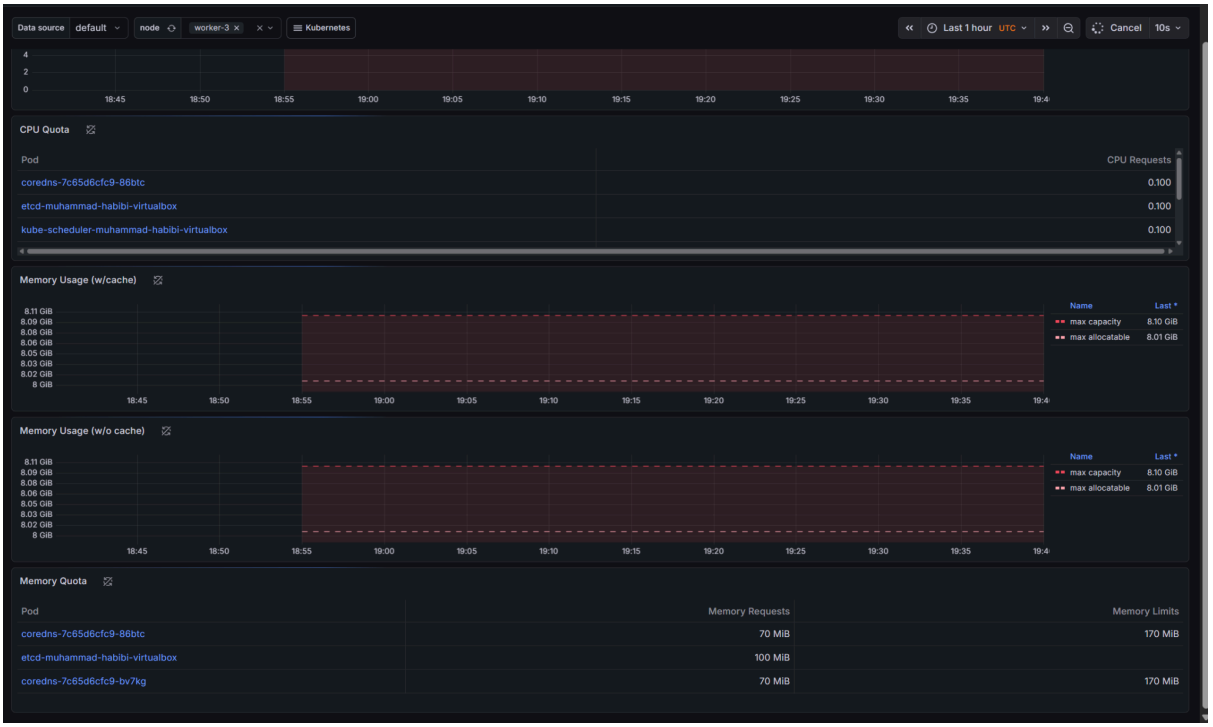
Gambar ini menampilkan halaman autentikasi Grafana yang berhasil diakses melalui peramban web eksternal. Keberhasilan akses ini mengonfirmasi bahwa *service* Grafana telah terekspos dengan benar menggunakan mekanisme *NodePort* atau *LoadBalancer*. Hal ini memastikan bahwa operator sistem dapat melakukan pengawasan visual terhadap metrik klaster secara *real-time* dari luar jaringan klaster.

Gambar 3. Visualisasi Metrik Penggunaan Resource pada Control Plane

Visualisasi dasbor Grafana ini menyajikan metrik penggunaan CPU dan memori secara spesifik pada mesin Control Plane. Data telemetri yang dikumpulkan oleh 'node-exporter' dan disimpan dalam pangkalan data deret waktu (*time-series*) Prometheus berhasil ditampilkan secara akurat. Pemantauan pada node ini kritikal untuk memastikan stabilitas komponen inti klaster seperti API Server, Scheduler, dan Controller Manager.

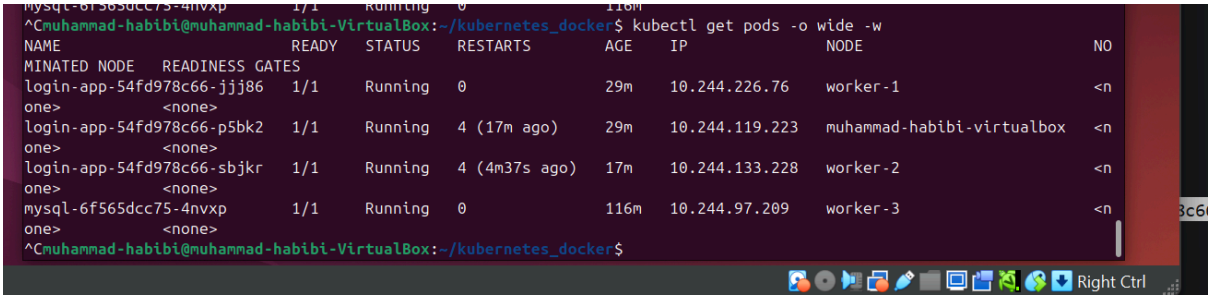
Gambar 4. Visualisasi Metrik Penggunaan Resource pada Worker-1

Gambar 5. Visualisasi Metrik Penggunaan Resource pada Worker-2



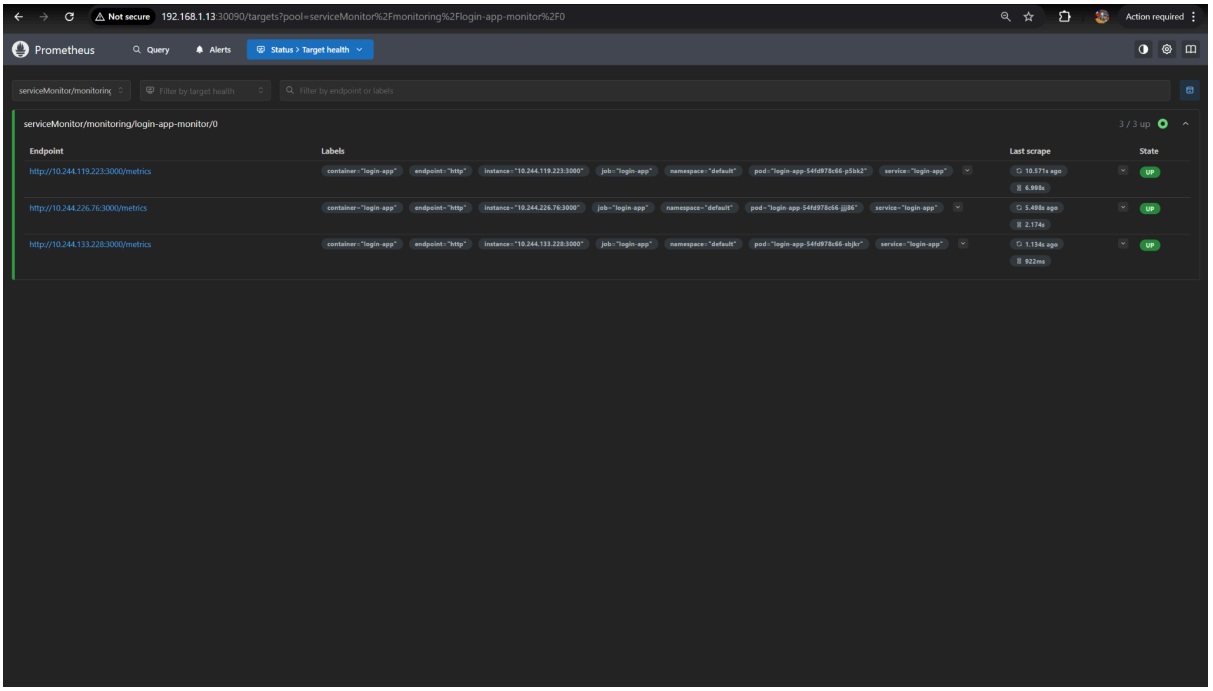
Gambar 6. Visualisasi Metrik Penggunaan Resource pada Worker-3

Rangkaian gambar ini mendokumentasikan pemantauan penggunaan sumber daya pada masing-masing *worker node* secara terpisah. Analisis terhadap grafik ini menunjukkan konsumsi memori dan CPU yang stabil pada setiap *node* pelaksana tugas. Kemampuan untuk memilah data metrik per *node* membuktikan efektivitas sistem *monitoring* dalam mendeteksi anomali atau ketimpangan beban kerja (*resource imbalance*) di seluruh infrastruktur klaster.



Gambar 7. Verifikasi Distribusi Pod Aplikasi login-app

Berdasarkan output ‘kubectl get pods -o wide’, terlihat bahwa tiga replika dari aplikasi ‘login-app’ telah dideploy dan aktif. Penempatan pod yang tersebar secara otomatis oleh *Kube-scheduler* ke Worker-1, Master, dan Worker-2 merupakan implementasi nyata dari prinsip *fault tolerance*. Jika salah satu node mengalami kegagalan, layanan tetap tersedia melalui replika di node lainnya.



Gambar 8. Status Target Service Monitor login-app pada Prometheus

Tampilan antarmuka 'Targets' pada Prometheus memvalidasi fungsionalitas ‘ServiceMonitor’ yang telah dikonfigurasi. Sistem berhasil melakukan penemuan layanan (*service discovery*) terhadap tiga titik akhir (*endpoints*) dari ‘login-app’. Status ‘UP’ pada seluruh target menandakan bahwa Prometheus sukses melakukan penarikan metrik (*scraping*) dari aplikasi untuk diolah lebih lanjut.

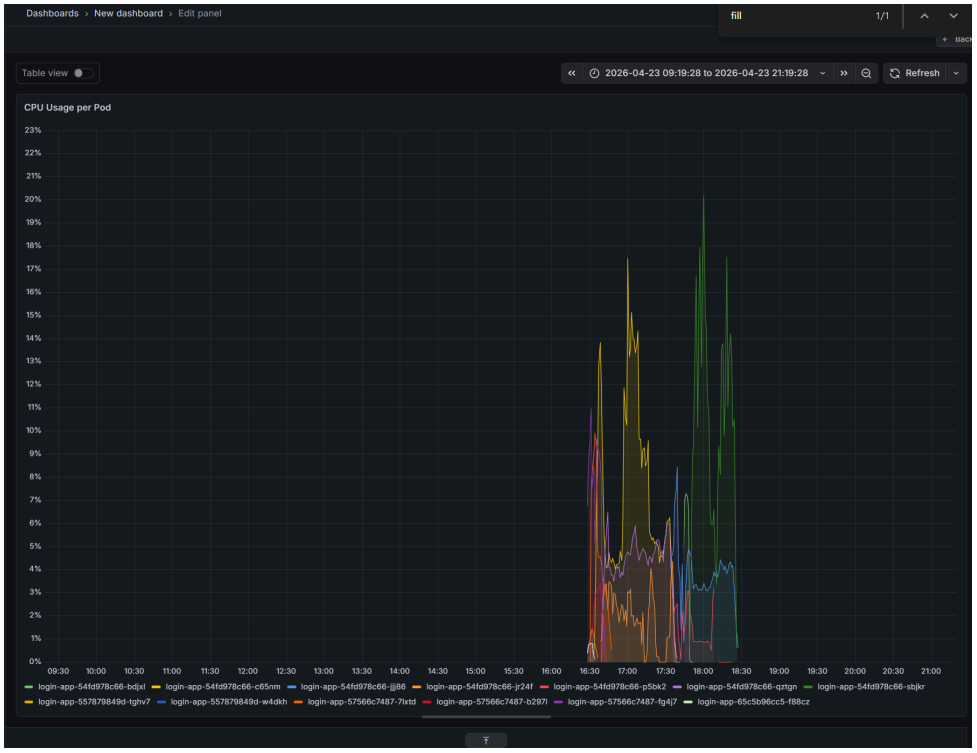


Table view
2026-04-23 09:19:28 to 2026-04-23 21:19:28
Refresh
Memory Usage per Pod
72 MiB
68 MiB
64 MiB
60 MiB
56 MiB
52 MiB
48 MiB
44 MiB
40 MiB
36 MiB
32 MiB
28 MiB
24 MiB
20 MiB
16 MiB
12 MiB
8 MiB
4 MiB
0 B
09:30 10:00 10:30 11:00 11:30 12:00 12:30 13:00 13:30 14:00 14:30 15:00 15:30 16:00 16:30 17:00 17:30 18:00 18:30 19:00 19:30 20:00 20:30 21:00
login-app-54fd978c66-bdjxl
login-app-54fd978c66-c65nm
login-app-54fd978c66-jjj86
login-app-54fd978c66-jr24f
login-app-54fd978c66-p5bk2
login-app-54fd978c66-qztgn
login-app-54fd978c66-sbjkr
login-app-557879849d-tghv7
login-app-557879849d-w4dkh
login-app-57566c7487-7lxtd
login-app-57566c7487-b297l
login-app-57566c7487-fg4j7
login-app-65c5b96cc5-f88cz

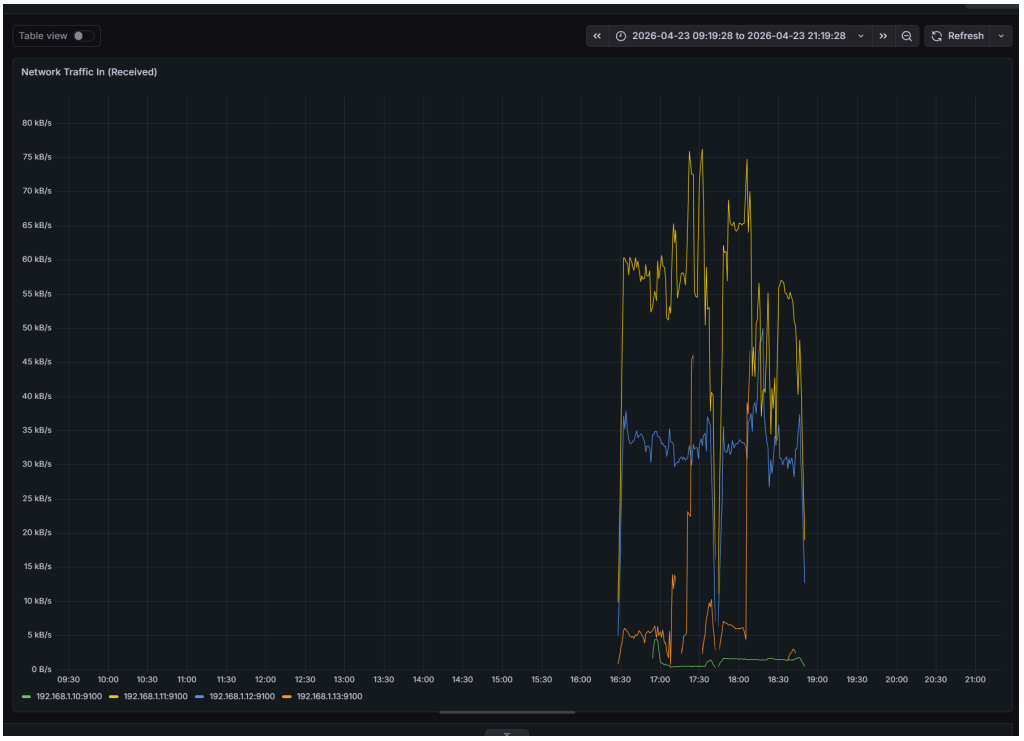


Table view
2026-04-23 09:19:28 to 2026-04-23 21:19:28
Refresh
Network Traffic In (Received)
80 kB/s
75 kB/s
70 kB/s
65 kB/s
60 kB/s
55 kB/s
50 kB/s
45 kB/s
40 kB/s
35 kB/s
30 kB/s
25 kB/s
20 kB/s
15 kB/s
10 kB/s
5 kB/s
0 B/s
09:30 10:00 10:30 11:00 11:30 12:00 12:30 13:00 13:30 14:00 14:30 15:00 15:30 16:00 16:30 17:00 17:30 18:00 18:30 19:00 19:30 20:00 20:30 21:00
192.168.1.10:9100
192.168.1.11:9100
192.168.1.12:9100
192.168.1.13:9100

Gambar 9. Dashboard Grafana untuk Analisis Kinerja Pod login-app

Dasbor kustom ini memberikan visibilitas mendalam terhadap performa spesifik pod ‘login-app’, meliputi konsumsi CPU, penggunaan memori, serta lalu lintas jaringan masuk dan keluar. Melalui visualisasi ini, pengembang dapat menganalisis efisiensi kode dan dampak operasional aplikasi terhadap infrastruktur secara granulasi, yang sangat penting untuk proses optimasi performa.

BAB 2 - Implementasi LB untuk Aplikasi yang Sudah di Deploy

```
muhammad-habibi@muhammad-habibi-VirtualBox:~/kubernetes_docker$ kubectl top nodes
NAME                        CPU(cores)   CPU%        MEMORY(bytes)   MEMORY%
muhammad-habibi-virtualbox  649m         12%         4350Mi          53%
worker-1                    248m         6%          1775Mi          46%
worker-3                    477m         11%         2009Mi          52%
worker-2                    <unknown>    <unknown>   <unknown>       <unknown>
muhammad-habibi@muhammad-habibi-VirtualBox:~/kubernetes_docker$ kubectl get hpa login-app-hpa
NAME            REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
login-app-hpa   Deployment/login-app   cpu: 27%/80%   2         4         3          49m
```

Gambar 10. Pengecekan Metrik Node dan Status Awal Horizontal Pod Autoscaler (HPA)

Pada tahap inisialisasi, dilakukan pengecekan beban awal menggunakan 'kubectl top nodes' dan pemantauan status Horizontal Pod Autoscaler (HPA). Kondisi awal menunjukkan beban CPU yang rendah (27%) dengan target ambang batas autoscaling yang ditetapkan pada 80%. Tahap ini sangat penting sebagai titik acuan (*baseline*) untuk menguji reaktivitas sistem otomatisasi terhadap lonjakan beban nantinya.

```
Swap:            0B          0B          0B
muhammad-habibi@muhammad-habibi-VirtualBox:~/kubernetes_docker$ kubectl get pods load-gen
NAME       READY   STATUS    RESTARTS   AGE
load-gen   1/1     Running   0          15m
muhammad-habibi@muhammad-habibi-VirtualBox:~/kubernetes_docker$ kubectl get pods -l app=login-app -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP              NODE                        NOMINATED NODE   READINESS GATES
login-app-57f64d66bf-6nmhj   1/1     Running   0          39m   10.244.226.79   worker-1                    <none>           <none>
login-app-57f64d66bf-drhck   1/1     Running   0          32m   10.244.119.225  muhammad-habibi-virtualbox  <none>           <none>
login-app-57f64d66bf-kb6k6   1/1     Running   0          54m   10.244.133.240  worker-2                    <none>           <none>
muhammad-habibi@muhammad-habibi-VirtualBox:~/kubernetes_docker$ kubectl get pods -l app=login-app -o wide -w
NAME                         READY   STATUS    RESTARTS   AGE   IP              NODE                        NOMINATED NODE   READINESS GATES
login-app-57f64d66bf-6nmhj   1/1     Running   0          40m   10.244.226.79   worker-1                    <none>           <none>
login-app-57f64d66bf-drhck   1/1     Running   0          32m   10.244.119.225  muhammad-habibi-virtualbox  <none>           <none>
login-app-57f64d66bf-kb6k6   1/1     Running   0          54m   10.244.133.240  worker-2                    <none>           <none>
```

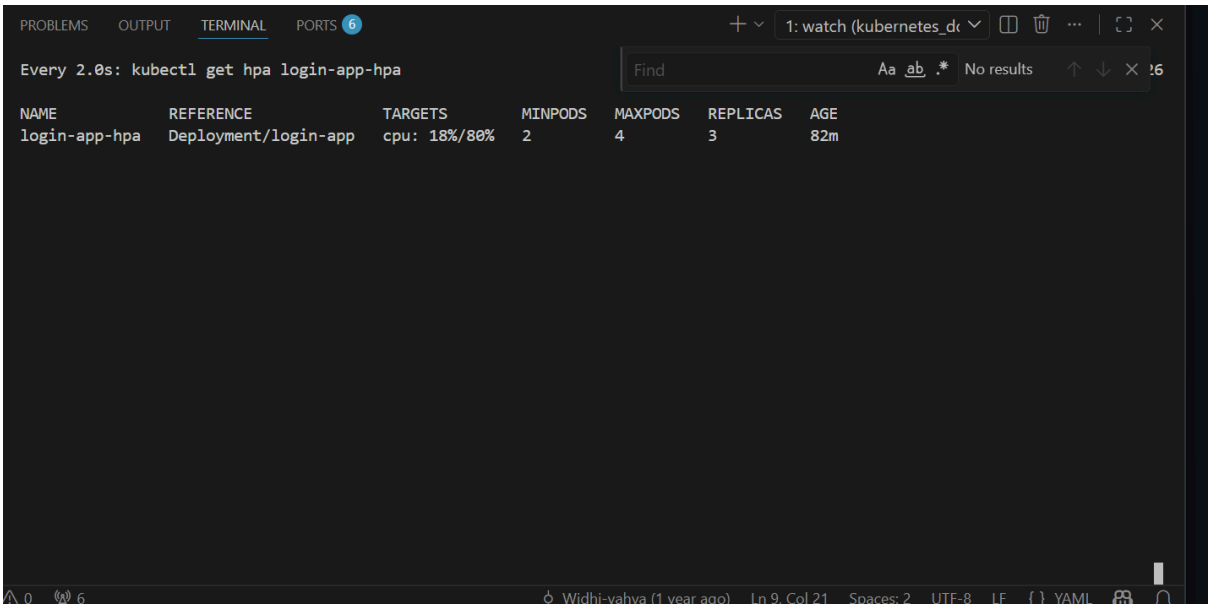
Gambar 11. Status Eksekusi Pod Load Generator terhadap Aplikasi

Dokumentasi ini memperlihatkan pengoperasian *pod* 'load-gen' yang dirancang untuk mensimulasikan lonjakan trafik pengguna secara intensif ke aplikasi 'login-app'. Pengecekan status pod secara berdampingan memastikan bahwa simulasi beban sedang berlangsung secara aktif sementara aplikasi sasaran terus memberikan layanan.

```
PROBLEMS   OUTPUT   TERMINAL   PORTS 6                    1: watch (kubernetes_d
Every 2.0s: kubectl get hpa login-app-hpa                Find   Aa ab .*  No results
NAME            REFERENCE              TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
login-app-hpa   Deployment/login-app   cpu: 87%/80%   2         4         3          79m
```

Gambar 12. Peningkatan Persentase Metrik CPU HPA Akibat Injeksi Beban

Gambar ini menangkap momen kritis di mana intensitas trafik dari ‘load-gen’ berhasil memicu lonjakan penggunaan CPU hingga 87%, melampaui batas aman 80% yang telah dikonfigurasi. Kondisi ini membuktikan bahwa HPA telah berhasil mendeteksi beban berlebih secara dinamis dan bersiap untuk menjalankan instruksi scale-out (penambahan kapasitas) guna menjaga stabilitas layanan.



Gambar 13. Penurunan Metrik CPU HPA Pasca Pengurangan Beban

Pasca penghentian simulasi beban, metrik penggunaan CPU terpantau menurun secara signifikan ke angka 18%. Hasil ini mendemonstrasikan kemampuan klaster dalam melakukan normalisasi mandiri. Sistem HPA menunjukkan stabilitas operasional dengan memantau penurunan beban sebelum nantinya melakukan *scale-in* (pengurangan replika) secara otomatis untuk mengefisiensikan penggunaan sumber daya.

LAMPIRAN

Link Github: https://github.com/nadhif-royal/k8s-monitoring-loadbalancing